Smart Secure Home

IoT-IPS Design Document

Architecture, Shared Contract, Module Reference, Incident Database, Threat Model and Evaluation

Graduation Project | Person 1: Architecture and Documentation

Version 1.0

Team members	Supervisor and institution
Ali Elsayed Samir Ziad Ayman Ziad Mahmoud Mariem Mohamed Mariem Amr Khaled Adel Zeyad Elwajeeh	Supervisor: Dr. Amr Megahed University: Egyptian Chinese University Faculty: Computer Science

Contents

Contents 2

1. Overview and Scope 3

2. System Architecture 4

3. Shared Contract 5

4. Device Profiles and Network Addresses 7

5. Risk Model (Person 4) 8

6. Detection Rules (Person 3) 9

7. Response Policy (Person 5) 10

8. Incident Database (Person 6) 11

9. Reports and Notifications (Person 7) 12

10. Data (Person 2) 12

11. Threat Model of the IPS Itself 14

12. Demo and Evaluation 15

13. Verification and Known Limitations 17

14. Package Contents and How to Run 17

1. Overview and Scope

This document is the design reference of the IoT-IPS. It defines how the seven modules fit together, the messages they exchange, the response policy, the incident database, the threat model of the system itself and the evaluation on a scripted demo. The IoT-IPS detects attacks against IoT devices, assesses their risk, decides on a proportionate response, records the incident, and explains it to a non-technical reader.

Target of this phase

A **CSV-to-report pipeline**: a network event goes in, and out come the attack type, confidence, risk score, action, incident record and readable reports. Detection is rule-based (an IPS without machine learning); a trained model replaces the detector next phase through the same input and output format.

In scope	Out of scope
CSV events built from CICIoT2023 flow features plus context fields Rule-based detector, risk engine, policy engine Simulated actions (dry-run firewall commands) Hash-chained SQLite incident database Technical, executive and notification texts Unit tests and one-command demo	Raspberry Pi hardware and a real camera Live blocking on a real firewall Live dashboard ML model training (next phase) LLM agent and prompt-injection defenses (when the agent is added)

Primary example device

The **IP camera** (CAM-001) is the main protected device in the examples and in the demo. No specific camera model has been chosen; its profile is updated when one is.

Result at a glance

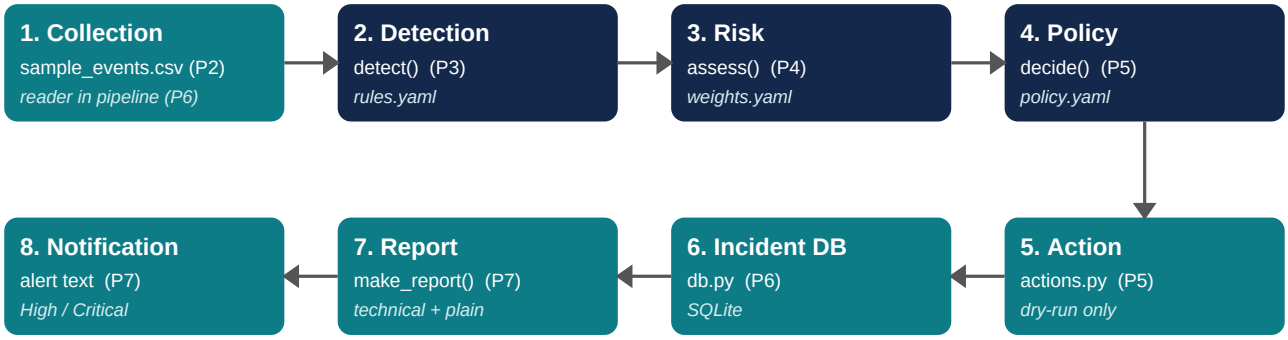
> The integrated demo replays **14 events** (4 benign, 10 attacks) through the real modules of Persons 3 to 7. It stores **7 incidents** (3 High, 4 Critical), detects **6 of 10** attacks with **1 false alarm** among 4 benign events, behaves as designed on **12 of 14** events, and the incident database passes its integrity check. All actions are simulated. Section 12 lists every event.

Team and ownership

Person	Owner	Role	Deliverables
1	Ali Elsayed Samir	Architecture and documentation	This document, devices.json, sample_events.csv (integrated), expected_results.csv
2	Ziad Ayman	ML data	train.csv, test.csv, features.txt, data_summary.md, sample events
3	Ziad Mahmoud	Rule-based detector	detector.py, rules.yaml, test_detector.py, RULES_EXPLAINED.md
4	Mariem Mohamed	Risk engine	risk.py, weights.yaml, test_risk.py, weight_comparison.md
5	Mariem Amr	Policy engine and actions	policy.py, policy.yaml, actions.py, ACTION_LOG_FORMAT.md
6	Khaled Adel	Pipeline and incident database	run_pipeline.py, db.py, run_demo.py, run_tests.py, incidents.db
7	Zeyad Elwajeeh	Reporting and notifications	report.py, templates, sample reports and notification texts

2. System Architecture

Every event travels through the same chain. Each stage is a separate module with one function and one message type in and out; thresholds, weights and policies live in YAML files that are validated when loaded.



Shared configuration: rules.yaml (P3) weights.yaml (P4) policy.yaml + allow-list (P5)
devices.json (P1, device profiles) Orchestration: run_pipeline.py and run_demo.py (P6)

Stage	Function	Input	Output	Owner
Collection	run_pipeline.read_events	sample_events.csv	Event (flat dict)	P2 / P6
Detection	detector.detect(event, device_profile)	Event	Detection	P3
Risk	risk.assess(detection, event, device_profile)	Detection, Event, profile	Risk	P4
Policy	policy.decide(risk, detection, event, device_profile)	Risk, Detection	Decision	P5
Action	ActionExecutor.run(decision)	Decision	action log record (dry run)	P5
Incident DB	IncidentDB.insert_incident(incident)	all messages	incident row (hash chained)	P6
Report	report.make_report(incident)	incident record	technical, executive reports	P7
Notification	notification text inside make_report	incident record	alert text for High and Critical	P7

Design principles

- **Loose coupling:** modules talk only through the messages in Section 3, so the rule-based detector can be replaced by a trained model without touching the other stages.
- **Configuration over code:** every threshold, weight and policy rule is in a validated YAML file.
- **Safe by default:** actions are dry-run, protected addresses are never blocked, and unknown actions are never executed.
- **Explainable:** the factor breakdown and the matching policy rule are kept with every incident, so a report can say why.
- **Tamper-evident:** incidents are stored in a hash chain that can be verified at any time.

Running it

`python run_pipeline.py run sample_events.csv --devices devices.json --reset` sends a CSV through the whole chain; `python run_demo.py` runs Person 6's demo and verifies the database; `python run_tests.py` runs every unit test. The pipeline looks for the other modules in this folder and in sibling folders named person2 to person7.

3. Shared Contract

Modules pass messages as plain Python dicts (JSON). The examples below are real: they follow one event, EVT-008 (300 failed SSH logins on the camera), through the chain.

3.1 Event (input)

An event is one flat dict. It holds the **context** fields written for each scenario and the **features**, which are CICIoT2023 columns taken from a real dataset row.

Context field	Type	Meaning	Used by
device_id	string	protected device the traffic belongs to (matches devices.json)	pipeline, risk, policy
timestamp	string	UTC, ISO 8601, ending in Z	pipeline, database, reports
src_ip	string	source IPv4 address	detector, risk, policy
dst_ip	string	destination IPv4 address	pipeline, reports
dst_port	integer 0-65535	destination port	detector, risk, reports
protocol	string	transport protocol, for example TCP, UDP, GRE	detector, reports
failed_attempts	integer	failed authentication attempts (logins)	detector, risk
connection_attempts	integer	connection attempts seen from the source in the window	detector, risk
unique_dst_ports	integer	distinct destination ports probed (used for port-scan detection)	detector
unique_dst_ips	integer	distinct destination hosts probed (used for scan detection)	detector
duration	string (optional)	how long the activity lasted, for example 4 minutes	reports

```
{"device_id": "CAM-001", "timestamp": "2026-10-21T09:05:15Z", "src_ip": "203.0.113.45", "dst_ip": "192.168.1.50",
 "dst_port": 22, "protocol": "TCP", "failed_attempts": 300, "connection_attempts": 300,
 "unique_dst_ports": 1, "unique_dst_ips": 1,
 "Rate": 150.86, "syn_count": 0.0, "SSH": 0.4, "TCP": 0.6, "Tot size": 207.1, "IAT": 0.01}
```

Feature columns keep their CICIoT2023 names (Rate, syn_count, SSH, TCP, Tot size, IAT and so on). Columns starting with `expected_` are for humans and are never passed to a module.

3.2 Detection (output of detect)

Field	Type	Meaning
label	BENIGN or ATTACK	detector verdict. BENIGN ends the chain: no risk score, no incident
attack_type	string	DoS, Brute Force, Port Scan or Unexpected Protocol/Port; the text None for a benign event. The risk engine normalizes the name
confidence	number 0 to 1	how sure the detector is (0.90 for a benign verdict)
mitre_technique	string or null	MITRE ATT&CK technique id from rules.yaml; null for a benign event

```
{"label": "ATTACK", "attack_type": "Brute Force", "confidence": 0.98, "mitre_technique": "T1110"}
```

3.3 Risk (output of assess)

```
{"score": 85, "level": "Critical", "weight_set": "balanced", "warnings": [],
 "factors": {
   "confidence":           {"value": 98.0,  "weight": 0.20, "contribution": 19.6,  "detail": "model_confidence"},
   "attack_severity":      {"value": 70.0,  "weight": 0.25, "contribution": 17.5,  "detail": "brute_force"},
   "device_criticality":   {"value": 75.0,  "weight": 0.20, "contribution": 15.0,  "detail": "high"},
   "persistence":          {"value": 100.0, "weight": 0.15, "contribution": 15.0,  "detail": "attempts=300"},
   "source_trust":         {"value": 90.0,  "weight": 0.10, "contribution": 9.0,   "detail": "external"},
   "behavioral_deviation": {"value": 100.0, "weight": 0.05, "contribution": 5.0,   "detail": "port_unexpected"},
   "potential_impact":     {"value": 75.0,  "weight": 0.05, "contribution": 3.75,  "detail": "brute_force"}}}
```

The score is an integer 0-100; the factor breakdown lets a report explain the result.

3.4 Decision (output of decide)

Field	Type	Meaning
action	list of strings	actions to carry out, in order, each defined in policy.yaml. Empty for a benign verdict
reason	string	which rule matched, with the level, score and confidence it used
command	list of strings	the firewall commands that would run (dry run); empty when nothing touches the network
dry_run	bool	always true in this version
rule_id	string	id of the matching rule
level	string or null	Low, Medium, High or Critical; null if the risk message was unusable
targets	object	src_ip, device_ip, device_id: validated values, or null
skipped	list	actions the rule asked for but a safety check removed: {action, why}
warnings	list of strings	fixed codes for bad or missing input

```
{"action": ["block_source", "isolate_device", "create_incident", "notify_admin"],
 "reason": "rule critical_certain matched: level Critical, score 85, confidence 0.98, attack known",
 "command": ["iptables -I FORWARD -s 203.0.113.45 -m comment --comment iot-ips -j DROP",
             "iptables -I FORWARD -s 192.168.1.50 -m comment --comment iot-ips -j DROP",
             "iptables -I FORWARD -d 192.168.1.50 -m comment --comment iot-ips -j DROP"],
 "dry_run": true, "rule_id": "critical_certain", "level": "Critical",
 "targets": {"src_ip": "203.0.113.45", "device_ip": "192.168.1.50", "device_id": "CAM-001"},
 "skipped": [], "warnings": []}
```

3.5 Conventions every module follows

Topic	Rule
Timestamps	UTC, ISO 8601, ending in Z
Risk level	capitalized: Low, Medium, High, Critical. Score 0-29 Low, 30-59 Medium, 60-79 High, 80-100 Critical

Topic	Rule
Attack names	the detector reports readable names (Brute Force, Port Scan, DoS, Unexpected Protocol/Port). The risk engine maps any spelling to its table keys (brute_force, port_scan, dos, unexpected_protocol, and so on); the database stores the normalized key
Confidence	0 to 1 in Detection; the risk engine converts it to 0-100
Incident id and status	id like INC-00001. Status is stored in lowercase: open, monitoring (Low incidents), contained, resolved; reports show it capitalized
Outputs	technical report, executive report (plain language for non-technical readers) and a short notification text for High and Critical
Missing or bad fields	never crash: use documented defaults and add a code to warnings
Untrusted text	every field coming from an event is untrusted: it is validated, length-limited and never passed to a shell

4. Device Profiles and Network Addresses

The risk engine reads a profile for the affected device. The camera is the primary device; the other profiles are examples used by the sample events and by the reference scenarios of the risk engine. Fields: `device_id, device_type, ip, criticality, expected_protocols, expected_ports`.

device_id	device_type	ip	criticality	expected_protocols	expected_ports	typical services
CAM-001	surveillance_camera	192.168.1.50	high	TCP, UDP	554, 443	RTSP, HTTPS, DNS
THERMO-001	thermostat	192.168.1.51	medium	TCP, UDP	53, 443	DNS, HTTPS
PLUG-001	smart_plug	192.168.1.52	low	TCP, UDP	80, 443, 53	HTTP, HTTPS, DNS
BULB-001	smart_bulb	192.168.1.60	low	TCP, UDP	80	HTTP, DNS
TEMP-001	temperature_sensor	192.168.1.70	medium	TCP	80, 1883	MQTT, HTTP
IND-001	industrial_sensor	192.168.1.80	critical	TCP	502	Modbus

- `expected_protocols` lists transport protocols, because the event's protocol field is TCP, UDP or GRE. The application services are listed in the last column for reference.
- Only `criticality` and `expected_ports` change the risk score: traffic to a port outside `expected_ports` raises the behavioral-deviation factor.
- The profiles are delivered in `devices.json`, an object with a "devices" list; the pipeline also accepts a plain list. Update the camera profile when a real camera is chosen.

Addresses used in this project

Role	Address	Defined in
IoT devices	192.168.1.50 to .52, .60, .70, .80	devices.json
Gateway and Raspberry Pi	192.168.1.1 (router), 192.168.1.2 (the Raspberry Pi)	allow_list in policy.yaml
Trusted machines (admin PCs)	192.168.1.10, 192.168.1.20, 192.168.1.30	trusted_ips in weights.yaml, allow_list in policy.yaml
Example attackers inside the LAN	192.168.1.66 (rogue laptop), 192.168.1.77	sample events, risk scenarios
Example outside addresses	203.0.113.x and 198.51.100.x, ranges reserved for documentation	sample events, risk scenarios

5. Risk Model (Person 4)

The risk score is a transparent weighted average of seven factors, each normalized to 0-100 and rounded to a whole number. All numbers live in `weights.yaml`, which `risk.py` validates strictly and refuses to run on a bad file. Three weight sets were compared; **balanced** is active.

Factor	balanced	device_centric	severity_centric	How it is computed
Detection confidence	0.20	0.15	0.25	confidence x 100
Attack severity	0.25	0.20	0.35	table by attack type (below)
Device criticality	0.20	0.30	0.10	low 25, medium 50, high 75, critical 100
Persistence	0.15	0.15	0.15	log scale of the larger of failed_attempts and connection_attempts: 1 is about 12, 10 about 42, 100 about 81, 300 or more is 100
Source trust	0.10	0.10	0.05	trusted 20, internal but unknown 60, external or invalid 90
Behavioral deviation	0.05	0.05	0.05	0 if dst_port is in the device's expected_ports, 100 if not; skipped (other weights re-scaled) if it cannot be decided
Potential impact	0.05	0.05	0.05	table by attack type (below)

attack_type	severity	impact	attack_type	severity	impact
port_scan	25	15	dos	75	70
reconnaissance	25	15	ddos	85	80
unexpected_protocol	35	35	mirai	95	95
spoofing	55	60	device_takeover	100	100
brute_force, web_based	70	75			

Why balanced was chosen

Measure (10 reference scenarios)	balanced	device_centric	severity_centric
Levels matched the human expectation	10 / 10	10 / 10	9 / 10
Gap Low to Medium / Medium to High / High to Critical	+29 / +14 / +4	+27 / +9 / +3	+23 / +12 / +6
Camera minus bulb, same attack (device awareness)	+10	+15	+5

- **severity_centric** mislabels a light brute force on a bulb as High and barely separates a bulb from a camera.
- **device_centric** separates devices best but has the thinnest margins, so small input changes could flip levels.
- **balanced** matches every scenario and has the widest margins at the lower levels.

Properties and limits

- Source trust can only nudge the score (weight capped at 0.15), because a source address can be spoofed.
- Missing or invalid data uses cautious defaults and is reported in `warnings` (for example unknown_attack_type, missing_source_ip, no_attempt_data).
- The High to Critical gap is small for every weight set (+4 for balanced), so the policy also looks at confidence rather than at the level alone.
- The maximum score is 99, not 100, for an outside source, because external counts as 90.
- The reference scenarios were adjusted while the engine was designed, so 10 / 10 is a calibration result, not independent proof.

6. Detection Rules (Person 3)

`detect(event, device_profile=None)` runs every rule on one event and returns the Detection message. Thresholds are in `rules.yaml`. Confidence equals the rule's base value when the measured value just reaches the threshold and rises linearly to the maximum at three times the threshold. When several rules fire, the highest confidence wins; a tie goes to the more severe rule (DoS, then Brute Force, then Port Scan, then Unexpected).

Rule	Fires when	Confidence	MITRE ATT&CK
DoS	Rate >= 1000 packets per second, or syn_count >= 100	0.80 to 0.99	T1498 Network Denial of Service
Brute Force	dst_port is 22, 23 or 554 (or the SSH or Telnet flag is 1) and failed_attempts >= 50 or connection_attempts >= 100	0.80 to 0.98	T1110 Brute Force
Port Scan	unique_dst_ports >= 20 or unique_dst_ips >= 10	0.75 to 0.97	T1046 Network Service Discovery
Unexpected Protocol/Port	dst_port is not in the profile's expected_ports, or protocol is not in its expected_protocols	0.60 (one mismatch), 0.75 (both)	T1571 Non-Standard Port

- Values from a CSV arrive as text; empty, NaN, infinite or non-numeric values count as missing. A bad event or a broken rule never stops the others.
- No rule fires: the verdict is BENIGN with confidence 0.90.
- The same function signature lets a trained model replace the rules.

Rules that stay quiet by design (evasion tests in test_detector.py)

Evasion	Why the rules miss it
Slow scan	fewer than 20 distinct ports in the window
Low-rate brute force	fewer than 50 failed attempts and fewer than 100 connections

Evasion	Why the rules miss it
Low-rate DoS	under 1000 packets per second and under 100 SYN packets
Attack on an allowed port and protocol	low volume on a port that the profile expects looks normal
Web attacks and spoofing	no rule covers them
Spread over many events	each event is judged alone, without memory

7. Response Policy (Person 5)

`policy.yaml` decides what happens for each risk level. Rules are checked top to bottom and the **first match wins**. The policy can only choose actions written in that file, and `policy.py` refuses to load a file that is unsafe (for example a Low or Medium rule that blocks, or an unknown action).

Rule	Matches	Actions
critical_certain	Critical, score >= 80, confidence >= 0.90, attack type known	block_source, isolate_device, create_incident, notify_admin
critical_probable	Critical, confidence >= 0.60	temporary_block, create_incident, notify_admin
critical_other	any other Critical	create_incident, notify_admin
high_probable	High, confidence >= 0.60	temporary_block, create_incident, notify_admin
high_other	any other High	create_incident, notify_admin
medium	Medium	log, create_incident, notify_admin
low	Low	log, create_incident, monitor
fallback (unusable_risk)	risk message missing or invalid	log, create_incident, notify_admin (never blocks)

Actions available (catalog in actions.py)

Action	Kind	What the simulation does
monitor, log	bookkeeping	records that the device is watched / records the event
create_incident	bookkeeping	tells the pipeline to store an incident
notify_admin	bookkeeping	records that the administrator must be told
temporary_block	network	would drop traffic from the source for 600 seconds
block_source	network	would drop traffic from the source until someone removes the rule
isolate_device	network	would drop all traffic to and from the protected device

Safety guards

- **Dry run only:** every network action only writes the exact command (and its undo command) to the action log. `dry_run: false` is rejected on purpose.
- **Confidence limits:** block_source and isolate_device need Critical, confidence 0.90 or more and a known attack type; temporary_block needs confidence 0.60 or more.
- **Allow-list:** 192.168.1.1 (router), 192.168.1.2 (Raspberry Pi), 192.168.1.10 (admin PC), 192.168.1.20 and 192.168.1.30 are never blocked, together with loopback, multicast, link-local and reserved addresses and the protected device itself.
- **Limits:** at most 256 active blocks and isolations; temporary blocks expire; the action log is rotated at 10 MB.

- **Checked twice:** the executor repeats the safety checks, so a tampered decision still cannot block the router.
- **No shell:** commands are built as argument lists from validated addresses.

The executor appends one JSON object per line to `actions.jsonl` (ts, seq, action, status, note, dry_run, device_id, rule_id, level, target, commands, undo, expires_in_s). Status is simulated, skipped or refused, and the note is a fixed code such as ok or source_is_device.

8. Incident Database (Person 6)

Incidents are stored in one SQLite table. Every rule of policy.yaml includes `create_incident`, so each non-benign event becomes an incident (Low included); a benign verdict stores nothing. Each row is linked to the previous one by a SHA-256 hash, so edited or deleted rows can be found.

Group	Columns
Identity	id, incident_id (INC-00001 style), created_at
Event	timestamp, device_id, device_type, src_ip, dst_ip, dst_port, attempts
Detection	attack_type, confidence
Risk	risk_score (0-100), risk_level (Low, Medium, High, Critical), risk_factors (JSON), risk_warnings (JSON list), weight_set
Response	action (JSON list), action_result (JSON list of action:status:note), reason, rule_id, dry_run (always 1)
Status	status: open, monitoring, contained or resolved. It is the only field that can change after insert
Reports	report_technical, report_plain (the executive report), report_notification (High and Critical only)
Integrity	prev_hash, row_hash

Rules and commands

- Parameterized SQL only. Every field is validated and length-limited before it is stored; an unusable score or level is rejected.
- The hash covers every field except status. Setting the environment variable `IOTIPS_DB_KEY` (16 or more characters) turns the chain into an HMAC that someone who can edit the file cannot recompute.
- The run prints the head hash; keeping it elsewhere and passing it to `verify --head` also catches deleted last rows.
- The database file is created with owner-only permissions; symbolic links are refused.
- CSV input limits: 20 MB, 100,000 rows, 300 columns; at most 10,000 incidents; a bad row is counted and skipped, and text from the CSV is never printed back in errors.

Command	What it does
python run_pipeline.py run sample_events.csv --reset	process a CSV into incidents.db and the action log
python run_pipeline.py show	table of stored incidents, count by level and by status (filters: --level, --device, --source, --attack, --status, --since, --limit)
python run_pipeline.py show --full INC-00003	one incident with its technical report, executive report and notification
python run_pipeline.py verify	checks the hash chain; exit code 1 if tampering is found (0 clean, 2 bad input or errors)
python run_demo.py / python run_tests.py	the whole demo / every unit test

9. Reports and Notifications (Person 7)

`make_report(incident)` builds three texts from templates (no language model). Every event field is treated as untrusted text and sanitized first.

Output	Audience	Content
Technical report	security analyst	9 sections: executive summary, technical indicators, threat analysis (MITRE ATT&CK), risk assessment with the factor table, response and containment (including the dry-run commands), recommended actions (immediate, short term, long term), governance mapping (NIST CSF 2.0, ISO/IEC 27001:2022), SHA-256 of the incident record, assumptions and limitations
Executive report	management, device owner	status banner, what happened, what the system did, whether the device is safe, business impact, what is needed from management, governance note, all in plain language
Notification	responsible personnel	short alert for High and Critical incidents: device, problem, risk, MITRE technique, action, status, response deadline, incident id

Risk level	Response target	Status shown in the report
Critical	15 minutes	Contained, Open or Monitoring, from the actions taken
High	1 hour	same
Medium	4 hours	same
Low	24 hours	Monitoring

Example notification (Critical)

```
CRITICAL IoT SECURITY ALERT

Device: CAM-001
Problem: Credential brute-force attempt from 192.168.1.100
Risk: 81/100 (Critical)
MITRE ATT&CK: T1110 Brute Force
Action: Source blocked; Device placed in quarantine; Administrator notified
Status: Contained
Respond within: 15 minutes
Incident: INC-00031
```

Sample reports, one per level, are delivered as INC-00028 (Low), INC-00029 (Medium), INC-00030 (High) and INC-00031 (Critical), plus INC-00032 (High, source on the allow-list). The pipeline generates the same three texts for every stored incident and keeps them in the database.

10. Data (Person 2)

Dataset: **CICIoT2023** (Canadian Institute for Cybersecurity), the provided CSV feature files rather than the PCAPs. Each row describes one window of packets with 39 numeric features; the 34 detailed classes are grouped into benign plus 7 attack groups.

Cleaning step	Rows left
Loaded (Merged01 part and per-attack files)	735,382
Removed rows with NaN or infinity (30)	735,352
Removed exact duplicates (33,608)	701,744
Removed identical values with both a benign and an attack label (656)	701,088
Removed identical values under a different attack label (3,774)	697,314

A balanced sample of **200,000 rows** (100,000 benign and 100,000 attack) was split 80/20, stratified by attack group: **160,000 training rows and 40,000 test rows**. After the features were chosen, rows that

repeat each other on the selected features were removed and each split was refilled, so **0 rows** share the same selected-feature values across train and test. Only part of the Merged01 file and of three per-attack files was downloaded, and every attack group is still present.

Group	Train	Test	Group	Train	Test
Benign	80,000	20,000	Spoofing	14,730	3,689
DDoS	14,732	3,689	BruteForce	10,437	2,609
DoS	14,731	3,689	Web	4,259	1,051
Mirai	6,381	1,584	Recon	14,730	3,689

Selected features

Eighteen features were chosen on the training part only: near-constant columns were dropped, the rest were ranked by Random Forest importance plus mutual information, and a column that is a near-copy (Spearman above 0.95) of one already chosen was skipped.

Group	Features (CICIoT2023 columns)	Why they help
Packet size	Max, Min, AVG, Variance, Tot sum, Header_Length	floods and scans send small, uniform packets; normal video and uploads reach full size
Volume and timing	Number, IAT, Time_To_Live	very many packets with near-zero gaps in DoS, DDoS and Mirai floods
TCP flags	syn_flag_number, ack_flag_number, psh_flag_number	many SYN packets without data mean a scan or a SYN flood
Protocol mix	TCP, UDP, HTTPS, HTTP, DNS, SSH	normal IoT traffic is mostly HTTPS and DNS; SSH share separates password guessing

Check on the 40,000 test rows (Random Forest, benign versus attack): 18 selected features give F1 0.8837, 6.31% false positives and 15.84% missed attacks; all 39 features give F1 0.8852. The selection keeps almost all of the accuracy with less than half of the columns. The detector of this phase does not use the trained model; the figures describe the data for the model of the next phase.

Context fields

CICIoT2023 has no device, time, address, port or attempt counts, so device_id, timestamp, src_ip, dst_ip, dst_port, protocol, failed_attempts, connection_attempts, unique_dst_ports, unique_dst_ips and duration are written for each sample event so that it tells a believable story.

Sample events

`sample_events.csv` holds 14 events: 4 benign and 10 attacks covering all 7 groups, including 2 evasion cases. Each event is a real row of the test part (never used for training) with its context. Person 2's 32 data checks pass on this file. For this integrated version, failed_attempts counts failed logins only, scans and floods carry unique_dst_ports and unique_dst_ips, and the port-scan event uses port 8080 as the last port probed.

11. Threat Model of the IPS Itself

An IPS that can block or isolate devices is also a target. These are the main ways it could be abused, with the mitigations built into the code and the limitations that remain.

Threat	How it works	Mitigation	Known limitation
Spoofed trusted source	Attacker forges an allow-listed or trusted address	Allow-listed sources are never blocked; source trust is capped at weight 0.15, so a spoofed trusted address lowers the score by about 7 points at most	A borderline Critical can become High with no block or isolation (sample report INC-00032: 77 High, block skipped). The incident is still recorded and the administrator notified
Spoofed victim	Attacker forges the address of a legitimate outside server (cloud or DNS) that is not on the allow-list, to get it blocked	Blocks at High are temporary (600 s); at most 256 active blocks	Add the addresses the devices depend on to the allow-list
Tampering with rules or config	Attacker edits rules.yaml, weights.yaml or policy.yaml to hide attacks or cause blocks	risk.py and policy.py validate their files strictly and refuse unsafe or malformed ones (duplicate keys, weights not summing to 1, Low or Medium rules that block, dry_run false); risk.py can check a SHA-256 of weights.yaml and warns if it is writable by everyone; Git review	Anyone with host access can still change files; the hash check must be switched on by the pipeline
Tampering with records	Attacker edits or deletes incidents or the action log	Hash-chained incident table (SHA-256, optional HMAC key); the verify command; head hash kept elsewhere; owner-only file permissions; action log rotated	Without the key or a saved head hash, a full rewrite of the chain is possible; active blocks are kept in memory and lost on restart
Flooding the pipeline	Huge number of events exhausts resources or floods the administrator	CSV limits (20 MB, 100,000 rows, 300 columns), at most 10,000 incidents, 256 active blocks, bad rows skipped, repeated blocks skipped as already blocked	Very large floods still degrade performance; no queueing in this phase
Slow attacks under thresholds	Slow scan or low-rate brute force stays below rule thresholds	Evasion tests document what is missed (EVT-006, EVT-009); the ML model of the next phase covers part of it	Threshold rules judge one event at a time and miss low-and-slow behavior
Compromised IoT device	A device such as the camera attacks from its own address	IoT device addresses are not on the allow-list or trusted list, so a device can be isolated (EVT-014); behavioral deviation raises risk	Adding a device address to either list would make it a trusted source
Malicious text in events and reports	Attacker-controlled strings injected into reports, logs or commands	Fields are validated and length-limited before storage; risk.py never copies free text; reports sanitize every field; commands are argument lists from validated addresses, no shell	LLM-specific defenses (prompt injection) are added when the agent is introduced
Wrong automatic isolation of the camera	A false positive cuts the camera off	Dry run only; isolation needs confidence 0.90 and a known attack; every command has a logged undo command; manual release	Some false positives are unavoidable; the demo has one (EVT-002)

12. Demo and Evaluation

The demo replays the 14 events of `sample_events.csv` through the real chain (detector, risk engine, policy, action executor, database, reports) with `python run_pipeline.py run sample_events.csv --devices devices.json --reset`. The expected outcome of each event was written from its story, before the run, in `expected_results.csv`: benign traffic should raise no alert, attacks that a rule covers should be detected at a sensible level, and attacks that no rule covers are expected to be missed (known limitations of Phase 1).

Event	Scenario	Expected	Detected as	Risk	Actions (create_incident always included)	Result
EVT-001	Owner's phone watches the camera stream	No alert	No alert	-	none	Pass
EVT-002	Camera uploads a clip to its cloud service	No alert	DoS, conf 0.80	61 High	notify_admin; temporary_block skipped (source is device)	Fail
EVT-003	Thermostat DNS lookup	No alert	No alert	-	none	Pass
EVT-004	Owner mistypes the plug password once	No alert	No alert	-	none	Pass
EVT-005	Port scan of 1,000 ports on the camera from an unknown laptop	Alert, Medium or High	Port Scan, conf 0.97	67 High	temporary_block, notify_admin	Pass
EVT-006	Slow scan of 15 ports on the camera	No alert (known limitation)	No alert	-	none	Pass
EVT-007	Vulnerability scan of the plug web page	No alert (known limitation)	No alert	-	none	Pass
EVT-008	300 failed SSH logins on the camera from an external address	Alert, Critical	Brute Force, conf 0.98	85 Critical	block_source, isolate_device, notify_admin	Pass
EVT-009	Low-rate SSH brute force, 6 guesses	No alert (known limitation)	No alert	-	none	Pass
EVT-010	SYN flood on the camera video port	Alert, High or Critical	DoS, conf 0.99	81 Critical	block_source, isolate_device, notify_admin	Pass
EVT-011	UDP flood on the camera	Alert, High or Critical	DoS, conf 0.99	86 Critical	block_source, isolate_device, notify_admin	Pass
EVT-012	ARP spoofing: a rogue device poses as the router	No alert (known limitation)	DoS, conf 0.99	67 High	temporary_block, notify_admin	Fail
EVT-013	Command injection on the plug web form	No alert (known limitation)	No alert	-	none	Pass
EVT-014	Infected camera floods an outside server (Mirai-style)	Alert, Critical	DoS, conf 0.95	82 Critical	isolate_device, notify_admin; block_source skipped (source is device)	Pass

Incidents stored by the run

Incident	Event	Device	Source	Attack	Conf	Score	Level	Status
INC-00001	EVT-002	CAM-001	192.168.1.50	DoS	0.80	61	High	open
INC-00002	EVT-005	CAM-001	192.168.1.66	Port Scan	0.97	67	High	open
INC-00003	EVT-008	CAM-001	203.0.113.45	Brute Force	0.98	85	Critical	open
INC-00004	EVT-010	CAM-001	203.0.113.45	DoS	0.99	81	Critical	open
INC-00005	EVT-011	CAM-001	203.0.113.45	DoS	0.99	86	Critical	open
INC-00006	EVT-012	CAM-001	192.168.1.66	DoS	0.99	67	High	open
INC-00007	EVT-014	CAM-001	192.168.1.50	DoS	0.95	82	Critical	open

Three of the 7 incidents are High and 4 are Critical. After the first block of 203.0.113.45 and the isolation of CAM-001, later events with the same target are logged as already blocked.

Detection quality on the 14 events

Measure	Result
Attacks detected (true positives)	6 of 10
Attacks missed (false negatives)	4 of 10: slow scan, vulnerability scan, low-rate brute force, command injection
False alarms (false positives)	1 of 4 benign events: the cloud upload (EVT-002)
Precision / recall / F1	0.86 / 0.60 / 0.71
Attack type correct among the 6 alerts	5 of 6: the ARP-spoofing row (EVT-012) is reported as DoS
Behaved as designed	12 of 14 events (the two failures are EVT-002 and EVT-012)

Both failures have one cause: the DoS rule fires at 1,000 packets per second, and the real feature rows of these two events carry Rate values of 1,039.6 and 4,366.8. The set is small and hand-chosen, so these figures show that the chain works and where it is weak; they are not a measurement of accuracy.

Final checks

Check	Result
Expected level, actions and incident for each event	12 of 14 as designed
Risk rises with severity	yes: the port scan scores 67, brute force and floods score 81-86
Incident records	7 incidents stored, each with a technical report, an executive report and a notification text
Dry run only	25 action-log records: 17 simulated, 8 skipped (already blocked or source is the device); no firewall changed
Database integrity	verify: chain is intact
Known misses documented	evasion tests in test_detector.py and Section 6

13. Verification and Known Limitations

Unit tests (run by their owners; counts from the files of this release)

Suite	Tests	Result	Remark
test_risk.py (Person 4)	54	all pass	
test_policy.py (Person 5)	51	all pass	
test_actions.py (Person 5)	38	all pass	
test_db.py (Person 6)	23	all pass	
test_detector.py (Person 3)	18	all pass	pytest-style functions; run with pytest
test_pipeline.py (Person 6)	43	37 pass, 6 fail	5 tests expect the placeholder detector's results; 1 passes None values to report.py
check_data.py (Person 2)	32	all pass	also on the integrated sample_events.csv

Known limitations

- **Detector coverage:** four rule families; web attacks, spoofing, slow scans and low-rate brute force are missed (4 of the 10 attacks in the demo).
- **Rate threshold:** 1,000 packets per second flags real benign traffic in the demo (EVT-002); it should be calibrated on the training data or replaced by the model of the next phase.
- **Device profile in the detector:** the pipeline calls detect(event) without the profile, so the Unexpected Protocol/Port rule does not run in the integrated demo.
- **Attack naming:** the detector tags unexpected traffic T1571, while the report module derives T1071 from the attack type.
- **Report wording:** reports describe actions as applied (for example Containment: Applied) although every action is simulated, and a skipped block still appears in the recommended actions (EVT-014).
- **Memory:** each event is judged alone; active blocks are kept in memory and lost on restart.
- **Evaluation size:** 14 hand-chosen events; the figures are indicative.

14. Package Contents and How to Run

The **Person 1** folder contains:

File	Purpose
IoT_IPS_Design_Document.pdf	this document
devices.json	the six device profiles used by the pipeline, the risk engine and the demo
sample_events.csv	the 14 integrated demo events (Person 2's real feature rows plus the context fields of Section 3.1)
expected_results.csv	the expected outcome of each event, written before the run
README.md	how to run the demo and the checks
Integration_Checklist.md	one-page list of the small changes that align each module with this design

Running the demo

```
project/
  Person 1/  IoT_IPS_Design_Document.pdf, devices.json, sample_events.csv, expected_results.csv
  person2/   data/ (train.csv, test.csv, features.txt, sample_events.csv)
  person3/   detector.py, rules.yaml
  person4/   risk.py, weights.yaml
  person5/   policy.py, policy.yaml, actions.py
  person6/   run_pipeline.py, db.py, placeholders.py, run_demo.py, run_tests.py
  person7/   report.py

cd person6
python run_pipeline.py run "../Person 1/sample_events.csv" --devices "../Person 1/devices.json" --db demo.db --reset
python run_pipeline.py show   --db demo.db      # incidents and counts by level
python run_pipeline.py verify --db demo.db      # hash chain check
python run_tests.py                             # every unit test
```

Needs Python 3.10 or newer and PyYAML. The pipeline finds the other modules in sibling folders named person2 to person7.